



Transcript

Week 5: Data and Feature Engineering

Video 1 - Data and Feature Engineering

Data is the backbone of modern decision-making, but raw data alone isn't enough. It must be collected, cleaned, structured, and transformed to unlock its full potential.

Data engineering is at the heart of managing and transforming raw information into meaningful insights. It involves collecting, storing, and preprocessing data to ensure its quality and usability. Feature engineering plays a crucial role in refining datasets for machine learning, enabling models to perform efficiently. From handling different data formats to applying techniques like scaling, encoding, and dimensionality reduction, these processes form the foundation of data-driven solutions.

By mastering these techniques, you can streamline workflows, enhance model accuracy, and tackle real-world challenges with confidence.

Are you ready to turn raw data into actionable intelligence?

Let's get started!

Video 2 - What is Data Engineering?

Data is the fuel that drives modern innovation, but without the right systems in place, it remains untapped potential.

Data Engineering is the foundation of data-driven decision-making. It automates data pipelines to move information seamlessly, collects and integrates data from multiple sources, and stores it in structured or unstructured formats. Engineers transform raw data into meaningful insights, optimize system performance for scalability, and ensure data quality. These essential functions make data accessible, reliable, and ready for analysis, enabling organizations to extract maximum value from their information.

Data engineering ensures smooth data movement from source to destination for analysis and decision-making. It starts with **Data Pipeline Development**, where automated workflows collect and transfer data from various sources to storage systems like data warehouses or lakes. **Data Integration** then unifies data from multiple sources, ensuring consistency and accuracy. **Data Storage Solutions** provide scalable systems like relational databases, No SQL, and data warehouses to handle large data volumes. **Data Transformation** processes clean, filter, and structure raw data for meaningful insights. To manage increasing data loads, **Scalability and Performance Optimization** ensures efficiency and speed. Finally, **Data Quality Management** continuously monitors and maintains data reliability for trusted decision-making. Together, these components create a powerful data infrastructure for organisations.

Data Engineering plays a critical role in managing data across its lifecycle. It starts with automated collection and ingestion, ensuring accuracy and consistency. Storage solutions like databases and data lakes provide secure and efficient data management. Processing and transformation convert raw data into structured formats, making it usable for



analysis. Data integration combines multiple sources into a unified view, while accessibility and security measures ensure compliance and safe usage. Finally, scalable infrastructure supports the ever-growing demands of data-driven businesses, making data engineering an essential component of modern analytics.

Data Engineers take on several key responsibilities to maintain a seamless data ecosystem. They develop automated pipelines that move data efficiently across systems. They extract and collect data from sources like APIs, databases, and IoT devices. Storage and management involve organizing data in warehouses, lakes, or cloud solutions. Processing and cleaning ensure data is structured, reliable, and analysis-ready. Integration merges data from different origins, while quality assurance maintains accuracy and consistency. Finally, scalability and performance optimization ensure systems handle increasing data loads effectively. These tasks form the core of data engineering, powering data-driven enterprises.

Beyond the core functions, Data Engineering extends into advanced areas that enhance security, accessibility, and efficiency. Engineers implement security and compliance measures to protect data from breaches and ensure regulatory adherence. A P I and access management provide controlled data retrieval for teams. Real-time processing supports immediate decision-making using live data streams. Archiving and retention strategies store historical data securely. Collaboration with data teams ensures analysts and scientists get the data they need, while technology management optimizes tools like Hadoop and Spark. Continuous improvement ensures data engineering evolves with changing business needs, making it a dynamic and ever-growing field.

How do you think data engineering will shape the future of technology and innovation?

Video 3 - What is Feature Engineering?

Ever wondered how machines learn from data? The secret lies in feature engineering—the process that turns raw data into meaningful insights, making machine learning models smarter and more accurate.

Feature engineering is a crucial step in the machine learning process. It transforms raw data into meaningful input features that enhance model performance. This involves selecting the most relevant features, modifying existing ones, and even creating new features to provide deeper insights into the data.

By encoding these features in a structured way, we make data more interpretable—not just for machine learning algorithms, but also for human analysis. Effective feature engineering improves model accuracy, reduces the risk of overfitting, and ensures that data is well-organised and comprehensible.

Ultimately, it's about extracting the most valuable information from raw data and presenting it in a way that allows machine learning models to make better predictions. This makes feature engineering an essential part of building robust and reliable AI systems.

Feature engineering plays a crucial role in preparing data for machine learning models. It involves selecting, transforming, and creating features to improve model performance. **Feature selection** focuses on identifying the most relevant variables while removing

irrelevant or redundant ones to enhance predictive accuracy. **Feature creation** generates new features from existing data, uncovering hidden patterns and relationships that strengthen the model's predictive power. **Feature transformation** modifies existing features to better suit the modeling process. This includes techniques like scaling, normalization, and aggregation to optimize data for machine learning algorithms.

Encoding categorical variables ensures categorical data is converted into numerical format using methods like one-hot encoding and label encoding, making it suitable for machine learning. **Handling missing data** involves addressing gaps in data by either filling missing values with estimates or applying techniques to manage them effectively during model training. **Dimensionality reduction** simplifies the dataset by reducing the number of features while preserving key information. Techniques such as Principal Component Analysis help decrease computational complexity and enhance model efficiency. Together, these techniques help structure raw data, making it more informative, efficient, and effective for machine learning models.

Feature engineering is essential in developing machine learning models, offering key benefits that enhance model performance. Well-engineered features significantly **improve predictive accuracy** by ensuring that models receive the most relevant and informative data. This leads to better performance and more reliable predictions. Feature engineering also helps **mitigate overfitting** by selecting and creating features that capture true underlying patterns rather than noise. As a result, models generalise better to new, unseen data. Additionally, transforming and creating new features **makes data more interpretable**, aiding both model training and human understanding. This enables more meaningful insights and informed decision-making. Carefully chosen features further **enhance model interpretability** by making outputs clearer and more transparent. This allows for better insights into data patterns and increases trust in model results. In summary, feature engineering plays a fundamental role in improving accuracy, reducing overfitting, and enhancing both data and model interpretability, leading to more actionable insights.

Data Engineering and Feature Engineering play distinct yet complementary roles in the data science ecosystem. Data Engineering focuses on designing and maintaining infrastructure for data collection, storage, and processing, ensuring efficient and scalable data management. It involves ETL pipelines, databases, and tools like Hadoop and Spark, benefiting data engineers and architects. On the other hand, Feature Engineering enhances raw data by selecting, creating, and transforming features to improve machine learning model performance. It focuses on making data suitable for modeling using tools like Pandas and Scikit-learn, primarily benefiting data scientists. While Data Engineering ensures a robust data pipeline, Feature Engineering refines data for accurate predictive analytics.

Strong data infrastructure and well-crafted features are the backbone of successful AI and analytics—are you making the most of both?

Video 4 - Data Engineering

In today's digital world, data comes in various forms—some neatly structured, some loosely organised, and others completely free-form. Understanding these types helps businesses manage, analyse and utilise data effectively.



Data can be categorized into three main types: Structured, Semi-Structured, and Unstructured Data. Structured Data is well-organized, stored in fixed formats like databases and spreadsheets, making it easy to search and analyze. It's commonly used in financial records, customer databases, and inventory systems. Semi-Structured Data doesn't follow a strict format but contains some organization through tags or metadata—examples include JSON, XML, and emails. This type of data is widely used in web logs, social media, and sensor data. Unstructured Data has no predefined format, making it harder to analyze but rich in information. It includes text documents, videos, images, and audio files, often found in customer reviews, social media content, and multimedia archives. While structured data is easiest to manage, semi-structured and unstructured data require advanced tools to extract insights.

Data is collected from various sources, each playing a critical role in storage, processing, and analysis.

Databases are structured collections that allow efficient data management and retrieval. They are classified into relational databases, SQL, which store data in structured tables with predefined schemas, and non-relational databases, No SQL, which offer flexibility for handling unstructured or semi-structured data.

S Q L databases like My SQL and Post GRESQL are widely used for complex queries and transactional consistency, while No SQL databases like Mongo D B and Cassandra are preferred for dynamic applications. These databases support business transactions, customer data storage, and inventory management.

API's, or Application Programming Interfaces, enable applications to communicate and exchange data. Restful API's use HTTP requests and are popular for their simplicity and scalability, while SOAP API's rely on XML-based messaging for secure and structured data transfer. A P I's play a key role in real-time data fetching, integrating third-party services, and accessing remote databases.

Web scraping is a technique used to extract data from websites through automated tools like BeautifulSoup, Scrapy, and Selenium. It is commonly employed for retrieving product prices, collecting news headlines, and analyzing social media trends. Businesses use web scraping for market research and competitive analysis, helping them track industry trends and customer sentiments.

Sensors collect and transmit data from physical environments, converting real-world parameters into readable formats. These include temperature sensors for monitoring environmental and industrial conditions, motion detectors for security automation, and GPS units for tracking location and movement. Sensors are widely used in smart cities for infrastructure management, in industrial monitoring to ensure equipment operates efficiently, and in healthcare for tracking vital signs and patient conditions.

By leveraging these diverse data sources, organizations can improve decision-making, enhance automation, and extract valuable insights from vast amounts of information.

Understanding data types and sources is essential for efficient data handling, integration, and informed decision-making, especially in AI and machine learning. Different types of data require specific storage, processing, and analysis methods, optimizing performance and ensuring accuracy. Effective data integration helps merge information from various sources, resolving format inconsistencies and improving quality. Choosing the right tools and technologies for data processing enhances efficiency and ensures reliable insights.



Recognizing data origins and structure improves accuracy by addressing inconsistencies and missing values. Strategic data utilization helps organizations leverage insights for competitive advantage, innovation, and smarter planning. Overall, a strong grasp of data sources and types supports better analytics, reliable AI models, and data-driven success.

Are you fully utilising your data's potential? Take action today to optimise, integrate and maximise data for smarter decisions and a competitive advantage!

Video 5 - Data Collection and Storage

How can organisations make sense of vast amounts of data? The key lies in effective **data collection and storage**, which ensures that valuable information is organised, accessible, and ready for analysis.

Efficient data collection is essential for ensuring accuracy and reliability in analysis. Various methods and tools help streamline this process, making data handling more systematic and effective. Two key approaches to efficient data collection are **Data Pipelines** and **ETL Processes**.

Data Pipelines automate the workflow of collecting, processing, and transferring data from different sources to storage systems. This reduces manual effort, minimizes errors, and ensures a smooth data flow. For example, **Apache Airflow** is a widely used open-source tool that helps manage complex data workflows. It enables users to define and orchestrate data pipelines efficiently. Similarly, **AWS Data Pipeline** automates data movement and transformation across AWS services, making data workflows more reliable.

ETL Processes Extract, Transform, Load involve a series of steps to gather raw data, convert it into a usable format, and load it into a storage system. This method ensures data is collected from multiple sources, cleaned, structured, and then stored for further analysis. Tools like **Apache Nee-fee** facilitate this process by providing an intuitive platform for designing data flows and managing data ingestion. **Talend** offers a suite of ETL tools for seamless data integration and transformation, while **Informatica** is known for its scalable and advanced data integration capabilities.

In summary, **data pipelines** help automate and streamline data movement, ensuring efficient transfer from source to storage. **ETL processes** manage data extraction, transformation, and loading to prepare it for analysis. Both methods are crucial for handling large volumes of data effectively, ensuring accuracy, and enabling better decision-making.

Choosing the right data storage solution is crucial for efficient data management, ensuring that information is accessible for analysis and decision-making. Different storage solutions cater to varying data types and analytical needs, offering scalability, flexibility, and performance.

Relational Databases are structured systems that store data in tables with predefined schemas, making them suitable for handling complex queries. They ensure strong consistency, integrity, and support transactional applications. For example, **My SQL** is widely used for its reliability and open-source flexibility. **Post gray SQL** offers advanced

features and extensibility, making it a strong choice for enterprise applications. **Oracle Database** provides comprehensive solutions for high-performance data management.

No SQL Databases are designed to handle unstructured or semi-structured data with flexible schemas, allowing them to scale horizontally. They are ideal for applications requiring high availability and handling diverse data types. **Mongo DB** is a popular document-oriented database that stores data in JSON-like formats, offering schema flexibility. **Cassandra** excels in distributed storage, ensuring high availability, making it suitable for large-scale applications. **Red is** is an in-memory key-value store known for its speed and efficiency in caching and real-time applications.

Data Lakes serve as repositories for raw data stored in its native format, offering scalability and flexibility for future analysis. These solutions can accommodate a wide range of data types, including structured, semi-structured, and unstructured data. **Amazon S3** is a highly scalable object storage service commonly used for storing and analysing large volumes of raw data. **Azure Data Lake Storage** integrates with the Microsoft ecosystem, providing a secure and scalable storage solution. **Hadoop Distributed File System or HDFS** enables high-throughput access to large datasets, making it a fundamental component in big data applications.

Data Warehouses are optimised for storing, querying, and analysing structured data. They integrate data from multiple sources to facilitate reporting, business intelligence, and data analytics. **Google BigQuery** is a fully managed data warehouse with fast SQL queries and built-in machine learning capabilities. **Snowflake** offers a cloud-based data warehousing solution that separates storage and compute resources for greater efficiency. **Amazon Redshift** is a high-performance data warehouse designed for fast querying and analytics of large datasets.

Each storage solution plays a crucial role in data management. The choice of the right storage system depends on factors such as data type, volume, query complexity, and scalability requirements. Selecting the appropriate solution ensures that data is efficiently managed, remains accessible, and supports business intelligence and analytics initiatives.

Selecting the right data storage solution ensures efficient management, accessibility, and scalability for analysis. Which storage solution best fits your data needs? Think about it!

Video 6 - Data Cleaning and Preprocessing

Messy, incomplete, or inconsistent data can lead to inaccurate insights and poor decision-making. How can you ensure your data is clean, reliable, and ready for analysis?

Data cleaning and preprocessing are essential steps in preparing data for analysis and modelling. These processes ensure that data is accurate, consistent, and properly formatted, enabling effective and reliable insights.

By cleaning and preprocessing data, errors, inconsistencies, and missing values are addressed, improving data quality. This step is crucial for building accurate models and drawing meaningful conclusions, ultimately leading to better decision-making.

Data cleaning is the process of identifying and correcting errors, inconsistencies, and inaccuracies in a dataset to ensure its quality. This step is crucial for preparing reliable

data for analysis or modelling, as poor-quality data can lead to misleading insights and inaccurate conclusions.

One of the primary tasks in data cleaning is handling missing values. Missing data can compromise the integrity of an analysis if not addressed properly. There are two main approaches to handling missing values: **imputation** and **removal**. Imputation involves filling in missing values using statistical methods such as the mean, median, or mode of a column, ensuring the dataset remains complete. Alternatively, removal eliminates records with missing values if their absence does not significantly impact the dataset, preventing potential bias in the analysis.

Another key step in data cleaning is **removing duplicate records**. Duplicate entries can create redundancy and distort analytical results. Identifying and eliminating these records ensures that each data point remains unique and that the dataset accurately reflects reality.

Correcting errors is also an essential part of data cleaning. Datasets can contain inaccuracies such as typographical errors or incorrect data formats. Fixing these errors involves **identifying and correcting misspellings**, ensuring that textual data is consistent. Additionally, enforcing **consistent formatting** helps maintain uniformity, such as standardising date formats to prevent inconsistencies in time-based data analysis.

Another critical aspect of data cleaning is **standardising data**. Ensuring consistency in formats, measurement units, and naming conventions improves data reliability and compatibility across different systems. Standardisation includes **converting units to a common scale**, such as ensuring that all weight measurements are recorded in kilograms instead of a mix of pounds and grams. It also involves **maintaining uniform naming conventions**, ensuring that categorical variables, such as product names or customer segments, follow a standardised format for consistency.

Python libraries such as Pandas provide powerful tools for data cleaning. For instance, the `df.dropna` function allows users to efficiently remove missing values, ensuring a complete and structured dataset. These automated tools simplify the data cleaning process, enabling analysts to focus on extracting meaningful insights rather than dealing with data inconsistencies.

By performing data cleaning and preprocessing, analysts ensure that datasets are free from errors, inconsistencies, and missing values. This process is fundamental for generating accurate insights, building reliable models, and making informed business decisions. Clean, structured data leads to more accurate predictions and better decision-making outcomes, ultimately enhancing the overall effectiveness of data-driven strategies.

Understanding the reasons behind missing values is crucial for addressing them effectively. Missing data can arise from various factors, each affecting the quality of the dataset differently.

One common cause is **data entry errors**, where mistakes occur during manual data entry or data collection, leading to missing values. For example, a user may forget to fill in a required field on a form, leaving the data incomplete.

Another reason is **incomplete records**, which happen when data collection processes fail to capture all relevant information. This is common in surveys where respondents skip certain questions, resulting in missing responses.

Data merging issues can also lead to missing values, especially when datasets from different sources do not align perfectly. Discrepancies between data structures may result in missing information when combining multiple datasets, such as customer data from different systems that lack corresponding entries.

System or process failures are another contributing factor. Technical issues, software bugs, or interruptions in data collection systems can prevent data from being recorded. For instance, a sensor malfunctioning during a specific period might lead to missing data points.

Lastly, **non-applicability** can cause missing values when certain features do not apply to all records. For example, in a dataset about vehicle specifications, the "sunroof" field may be missing for cars that do not have one, as this feature is irrelevant to those vehicles.

By identifying the underlying causes of missing values, analysts can apply appropriate strategies to handle them, such as imputation, removal, or alternative data collection methods. Addressing these issues ensures that datasets remain robust, accurate, and reliable for analysis and decision-making.

Handling missing values is a crucial step in data preparation, as incomplete data can lead to inaccurate analysis, unreliable models, and flawed decision-making. To address this issue effectively, various techniques can be applied based on the type of data, the reason for the missing values, and the extent of incompleteness in the dataset.

One of the simplest approaches is **removal**, which involves deleting records containing missing values. This method is most effective when the proportion of missing data is small, ensuring that its removal does not significantly impact the dataset. By eliminating incomplete records, the remaining data becomes more structured, reliable, and easier to analyse. However, this approach is not suitable when a large portion of the dataset has missing values, as excessive removal can lead to loss of valuable information.

Another widely used technique is **imputation**, where missing values are replaced with estimated values to maintain dataset completeness. There are several imputation methods, each suited to different data types. **Mean, median, or mode imputation** replaces missing values using statistical measures based on the distribution of existing data. This method is particularly useful for numerical datasets where missing values occur randomly and do not follow a specific pattern.

For time series or ordered data, **forward and backward filling** is a common imputation technique. **Forward filling** replaces missing values with the last available value, ensuring that gaps are filled using previously recorded information. In contrast, **backward filling** uses the next available value to populate missing entries. These methods are useful for datasets where values change sequentially over time, maintaining data continuity.

More advanced imputation techniques include **K-Nearest Neighbours Imputation**, which estimates missing values based on the values of similar records. This method identifies patterns in the dataset by comparing records with similar characteristics and using them to infer missing information. KNN imputation is particularly useful when there is a strong relationship between features, as it ensures that imputed values are contextually relevant.

Another sophisticated method is **Regression Imputation**, which uses regression models to predict missing values based on other available features in the dataset. By analysing correlations between variables, regression imputation generates estimated values that align with existing data trends. This approach is highly effective when missing values follow a predictable relationship with other attributes in the dataset.

Interpolation is another powerful technique for estimating missing values, particularly in time series data. **Linear interpolation** assumes that values change at a constant rate over time, filling in gaps by drawing a straight line between known data points. This method ensures a smooth transition between values and provides a reliable estimate of missing data points based on observed trends.

In some cases, missing values do not need to be explicitly handled, as certain **machine learning algorithms** can process incomplete data internally. **Decision trees and random forests**, for example, can work with datasets that have missing values by making decisions based on available information rather than requiring imputed values. This approach simplifies data preparation and is especially beneficial in scenarios where imputation may introduce bias or uncertainty.

Selecting the most appropriate technique for handling missing values depends on the nature of the data, the reason for missing entries, and the impact of imputation or removal on the dataset. By applying the right strategy, analysts can ensure that their data remains complete, accurate, and ready for meaningful insights and robust decision-making.

Duplicate records often occur when merging datasets, manually entering data, or due to system glitches. These duplicates can lead to redundancy, skew analysis, and impact data integrity.

One common cause of duplicates is merging data from multiple sources. When datasets from different systems are combined, the same data may be recorded multiple times, leading to duplicate entries. Another frequent cause is repeated data entry, where manual processes result in the same information being recorded more than once due to human error or oversight. System glitches can also introduce duplicate records when errors occur during data capture, transmission, or storage.

To handle duplicates effectively, a common method is dropping duplicate entries, which involves removing redundant records while retaining the first or last occurrence. This can be done efficiently using Python's **D-F dot drop underscore duplicates** function. Depending on the analysis requirements, you can choose to keep the first occurrence using **D-F dot drop underscore duplicates, keep equals first** or the last occurrence using **D-F dot drop underscore duplicates, keep equals last**.

By identifying and removing duplicates, datasets remain clean, accurate, and optimized for analysis, ensuring reliable insights and improved model performance.

Outliers are data points that significantly deviate from the rest of the dataset. These extreme values can have a major impact on statistical analysis and machine learning models, potentially distorting results and reducing model accuracy. Detecting and managing outliers is crucial to ensuring that data remains accurate and reliable.

One of the main concerns with outliers is **data integrity**. Outliers can distort averages, skew distributions, and create misleading insights. If left unaddressed, they may result in inaccurate conclusions, affecting decision-making.

Outliers also impact **model performance**, particularly in machine learning. Many algorithms, such as linear regression and k-nearest neighbours, are sensitive to extreme values. These models can be thrown off by outliers, leading to poor generalisation and reduced predictive accuracy. Identifying and handling outliers helps ensure that models learn from relevant data without being influenced by extreme values.

Another key factor is **resource optimisation**. When outliers are not properly managed, they can waste computational resources by drawing unnecessary attention to irrelevant data points. By detecting and addressing outliers, analyses become more efficient, allowing computational power to be used effectively.

Properly identifying and handling outliers is an essential step in data preprocessing, ensuring clean, accurate, and optimised data for meaningful analysis and model training.

Detecting outliers is essential for ensuring data integrity and improving the accuracy of statistical analyses and machine learning models. There are two primary approaches to identifying outliers: **statistical methods** and **visualization techniques**.

Statistical methods use numerical measures to detect outliers. One common approach is the **Z-score**, which measures how far a data point deviates from the mean in terms of standard deviations. Typically, a Z-score above **three** or below **negative three** is considered an outlier, as it indicates a significant deviation from the rest of the data. In Python, this can be calculated using **SciPy dot stats dot Z-score of data**.

Another statistical approach is the **Interquartile Range, or IQR**, method. This method calculates the spread between the first quartile, **Q1**, and the third quartile, **Q3**. Any data point that falls beyond **one point five times the IQR** from **Q1 or Q3** is considered an outlier. In Python, this can be implemented using **IQR equals Q3 minus Q1**, and filtering values outside the range of **Q1 minus one point five times IQR** and **Q3 plus one point five times IQR**. This method is especially useful for skewed distributions where extreme values do not follow a normal pattern.

In addition to statistical techniques, **visualization methods** help in spotting outliers effectively.

One widely used tool is the **box plot**, which displays the distribution of data, highlighting potential outliers as individual points beyond the "whiskers" of the plot. The whiskers extend to the smallest and largest values within 1.5 times the I-Q-R, making it easy to identify extreme values. In Python, a box plot can be created using **P-L-T dot boxplot of data** in Matplotlib.

Another powerful **visualization** tool is the **scatter plot**, which represents individual data points on a graph. Outliers can be visually identified as points that are far removed from the main data cluster. This technique is particularly useful for analyzing relationships between variables and spotting anomalies. A scatter plot can be generated in Python using **P-L-T dot scatter of X and Y**, allowing users to detect deviations from expected patterns.

By combining statistical methods and **visualization techniques**, analysts can effectively detect and manage outliers, ensuring that data remains clean, reliable, and optimized for analysis and modeling.

Managing outliers is essential to ensuring accurate data analysis and reliable model performance. Outliers that are erroneous or irrelevant can distort results and should be handled appropriately. There are two primary approaches: removal and transformation.

Removing outliers is useful when extreme values do not represent genuine data points but instead result from errors or anomalies. These outliers can be safely eliminated to prevent them from skewing analysis. One common approach is filtering the dataset based on calculated thresholds, such as using a lower and upper bound to identify values that fall outside a reasonable range. In Python, this can be done using the command shown, which retains only the relevant data points.

In cases where outliers need to be retained but their impact should be minimised, **data transformation** is a useful alternative. Log transformation is an effective technique for reducing the influence of large outliers by applying a logarithmic scale to compress the data distribution. This method helps normalise the dataset, making it more uniform and preventing extreme values from dominating the analysis. It is particularly useful for positively skewed data, where a few large values disproportionately affect the results.

Another approach is **Winsorisation**, which limits extreme values by capping them at a specific percentile rather than removing them. Instead of eliminating outliers, Winsorization adjusts their values so they are closer to the rest of the dataset, reducing their impact while preserving their presence in the analysis. This technique is particularly beneficial when outliers contain meaningful information but need to be controlled to prevent distortion in the results.

By applying appropriate outlier management techniques, analysts can ensure that data integrity is preserved, leading to more reliable insights and accurate model predictions.

Properly managing outliers improves data accuracy and enhances model performance.

How do you handle outliers in your analysis?

Video 7 - Case Study: Titanic Dataset

The Titanic disaster is historic—but can data predict who had the highest chances of survival? Let's analyse the Titanic dataset to uncover key insights!

The Titanic dataset is widely used in data science for exploratory analysis and machine learning. It contains information about passengers aboard the RMS Titanic, which sank after hitting an iceberg on April 15, 1912. This dataset includes key details such as passenger demographics, ticket information, and survival status, making it valuable for predictive modeling. Researchers use it to analyze factors that influenced survival and gain data-driven insights.

The dataset contains several important features. Passenger I.D. is a unique identifier for each passenger. Survived indicates whether a passenger lived or did not survive, represented by one for survived and zero for not survived. P-class, or passenger class, represents ticket class: one for first class, two for second class, and three for third class. Name includes the passenger's full name and titles. Sex records whether a passenger is



male or female. Age represents the passenger's age in years, though some entries may be missing.

Additional features provide insights into family relationships and ticketing details. Sib SP, short for siblings and spouses, records how many siblings or spouses a passenger had aboard the Titanic. Parch, short for parents and children, represents the number of parents or children aboard. Ticket stores the ticket number, while Fare indicates the price paid for the ticket. Cabin records the cabin number where the passenger stayed, though many entries are missing. Embarked shows the port where the passenger boarded the Titanic, with three values: C for Cherbourg, Q for Queenstown, and S for Southampton.

By analyzing these features, we can identify survival patterns and explore the impact of class, age, and fare on survival outcomes.

The Titanic dataset provides valuable insights into passenger demographics, ticket details, and survival outcomes. This snapshot of the dataset displays key features such as passenger ID, survival status, class, name, gender, age, and family relations. It also includes ticket numbers, fares paid, cabin assignments, and the port of embarkation. By analysing this data, we can identify trends and factors that influenced survival rates on the Titanic.

To begin basic data cleaning, first, import the necessary libraries. Here, we use the Pandas library, which is essential for data manipulation and analysis in Python. Next, load the dataset using the `read_csv` function from Pandas. This function helps convert a CSV file into a DataFrame, making it ready for further processing. Finally, view the first few rows of the dataset using the `head` function to check the data structure.

To check for missing values in the dataset, use the `isnull` function. This function identifies null values in each column. The `sum` function then aggregates the total count of missing values per column. Identifying these missing values is crucial for deciding on the appropriate data cleaning approach.

Handling missing values is a crucial step in data preprocessing. In the 'Age' column, we use the `fillna` function along with the median value to replace missing data. The median is chosen over the mean because it is less sensitive to outliers, making it a more reliable measure of central tendency when extreme values are present. This ensures that the filled values better represent the actual distribution of the data.

For the 'Embarked' column, missing values are replaced with the most frequently occurring value, also known as the mode. The `mode` function identifies the most common entry in the column, which is then used to fill the missing entries. This approach is particularly useful for categorical variables, as it ensures that the replaced values align with the majority of the dataset. Using the mode for categorical data, like embarkation points, helps maintain consistency and improves the dataset's overall reliability for further analysis.

In data preprocessing, removing unnecessary columns helps streamline the dataset and improve analysis. Here, we drop the 'Cabin' and 'Ticket' columns using the `drop` function in Python.

The 'Cabin' column is removed because it contains too many missing values, making it difficult to impute or extract meaningful insights. The 'Ticket' column is dropped due to its irrelevance—it likely contains unique or near-unique values that do not contribute to predictive modeling and may introduce unnecessary noise.

By eliminating these columns, we simplify the dataset, ensuring that only the most relevant features remain for meaningful analysis and accurate predictions.

Ensuring that a dataset contains only unique records is crucial in data preprocessing, as duplicates can distort analysis. The `drop_duplicates` function helps identify duplicate rows by returning a boolean series, where `True` indicates a duplicate. The `sum` function then aggregates the results to count the total number of duplicates in the dataset.

Once identified, duplicates can be removed using the `drop_duplicates` function. By setting `inplace=True`, the operation is applied directly to the dataset, ensuring that only unique records remain. Removing duplicates helps maintain data integrity and prevents biased or misleading analysis results.

Outliers can significantly affect the accuracy of data analysis and machine learning models. Identifying and handling them is a crucial step in data preprocessing.

One effective method for detecting outliers is the Interquartile Range, or IQR. The IQR represents the middle 50 percent of the data, calculated as the range between the first quartile $Q1$ and the third quartile $Q3$. Any value below $Q1 - 1.5 \times \text{IQR}$ or above $Q3 + 1.5 \times \text{IQR}$ is considered an outlier.

The `detect_outliers` function calculates $Q1$, $Q3$, and the IQR for a given column. It then determines the lower and upper bounds and flags any values outside this range as outliers.

In this example, the function is applied to the `Age` and `Fare` columns. It creates two new columns, `Age Outliers` and `Fare Outliers`, which store boolean values—`True` for outliers and `False` for non-outliers. These indicators help identify extreme values that may require further analysis or adjustment.

Visualising outliers helps in understanding their impact on data distribution. One effective way to do this is by using box plots, which highlight the presence of extreme values.

In this example, we plot the distribution of the `'Age'` and `'Fare'` columns before and after removing outliers. The first box plot for each column displays the data with outliers included, while the second box plot shows the distribution after filtering out outliers.

By comparing these visualisations, we can see how outliers influence the spread of data and make more informed decisions about handling them in further analysis.

This visualisation compares the distribution of `'Age'` and `'Fare'` before and after removing outliers using box plots.

The left box plots display the original data, where extreme values are visible as points outside the whiskers. The right box plots show the cleaned data, with outliers removed, resulting in a more compact distribution.

By examining these plots, we can see how outliers impact the range and spread of data. Removing them can improve model performance and ensure more reliable analysis.

After identifying outliers in the `Age` and `Fare` columns, the next step is to remove them. This ensures the dataset used for further analysis or modeling is free from extreme values that could skew results.



To do this, a new DataFrame called `df_underscore_cleaned` is created, keeping only the rows where Age Outliers and Fare Outliers are false. The tilde symbol inverts the boolean condition, filtering out records that contain outliers in either column. As a result, `df_cleaned` excludes all records with outliers, producing a more refined dataset.

Once the outliers are removed, it is important to verify the changes by summarizing the cleaned dataset. The `describe` function provides key summary statistics, including count, mean, standard deviation, minimum and maximum values, and quartiles. Reviewing these statistics confirms that extreme values have been successfully removed, leading to a more balanced and reliable dataset for further analysis.

By effectively handling outliers, data quality improves, resulting in more accurate analysis. How will you apply these techniques to enhance your data preprocessing workflow?

Video 8 - Feature Engineering- Converting raw data into insights?

Welcome to our session on feature engineering, Extracting value from raw data.

Feature engineering is often called the secret sauce of machine learning.

While models and algorithms tend to get the spotlight, it's the features crafted and selected with care that truly power strong performance.

In fact, feature engineering is the most important component of your entire machine learning project life cycle that you talk about.

We build models, we check the performance and more often not when the model is not doing very well, we come back at the feature engineering stages and try to see what else we can do.

A strong foundation and feature engineering technique will help you to build a very robust model.

So far you have built a solid foundation.

You have mastered the essentials of data engineering from collection to cleaning.

You have explored the role of features in shaping model performance and through the tightening data set you have seen real world impact.

Like in the tightening data set, you had a feature which was passenger class and our outcome was survived.

We wanted to know if this feature has any impact on survival or not.

And based on this association that we are trying to understand over here, we can decide what feature to keep and what to drop when we want to predict our outcome as survival.

For example, when you talk about passenger ID, passenger ID does not play any role whatsoever in explaining survival column.

Then why should I keep into my consideration just an ID?



Artificial Intelligence and Machine Learning

I can drop it and then gradually I will check all the other features as well if it has some kind of relationship with the survival or not, and if I don't find any kind of relationship, I will drop that.

This is a simple process we follow to carefully select the features which are important in order to explain your outcome or it is important from your analysis perspective.

So we are talking about choosing the right features.

We are talking about how systematically we can go and select the right features from your given data set.

Your data set might have hundreds of features given to you, but based on the objective, you need to decide what makes sense, what does not make sense from the analysis perspective.

But why do we want to do that?

One, it will reduce the computation.

For me if I come to know from the given 100 features, only 70 of them are important, I will reduce the computation time significantly.

Another problem that we have in most of the machine learning model is if you have too many features, model tend to overfit.

Means model work well on the training data.

It fails to generalize from any unseen data.

That's why it's important that you choose the right features and for that, it's important that you understand how to perform EDA in detail and also what are the techniques which can help you to find the relationship or association between the given set of features and your outcome.

The common techniques that we follow is your visualization.

Like in the case of Titanic data, we wanted to know if being male or female has any impact on survival.

We can simply do a bar plot and I can understand that.

Or sometimes you create new features from the given set of features because creating a new feature might give you a better explanation of the outcome variable.

Like in the Titanic data, you have siblings, spouse, you have parent, children and you can combine these two features to create a new feature called family size which might give you a better explanation on survival.

This also gives you the idea whether the person was travelling alone or with somebody and that might again help you to understand being alone or being with family has any kind of extra significance or extra significant impact on the outcome.



Another important aspect of feature engineering is transforming existing feature.

Now what do I mean by that?

Let's try and understand.

Let's explore a simple but powerful example.

Suppose you are predicting how long a task takes to complete and a key feature is server processing power measured in gigahertz.

If you look at the data over here, we have been given different server power and for different server power, what was the processing time?

Now we want to work with this data, build a model and come up with the solution where I can enter server power and it can tell me what would be the processing time that the common model that we build for.

This is called linear regression model.

Now what linear regression does it?

It quantifies the relationship between two numerical variables and it models the linearity between two variables.

It actually creates a line which can explain the relationship.

Now if you look at the relationship between these two columns here, this is a scatter diagram between your server power and processing time.

This is how the relationship is.

And if I'm trying to build a linear model, what it does is, as I mentioned, it tries to fit a line over here, which can explain this relationship.

And if you could look at the line over here, it does a good job.

But still some of the points are quite far from the line.

And looking at it, we can sense that this is not a linear relationship, this is more of a curvilinear relationship.

Now how transforming a feature might help us?

So next what we do here is instead of working with X which has been given to us as server power, we get one by X as input feature.

So moment X is given to us, we calculate a new feature which is simply $1 / X$.

Why am I trying this particular transformation?



Because in this situation it works.

What other other transformations we can do?

We'll discuss that.

But once I did that, I realized now I get a new relationship over here, which is if you look at it more linear compared to what we had earlier.

So when I transform the feature, my relationship has become more linear and my linear regression model will do much better job on this compared to the non transformed data and I'll get more accurate model.

So simple transformation like this gives me much better result and that's why transformation transformation is very important.

But what kind of transformation we can do?

We can take X and we can say make it X square, make it X cube or make it $1/X$ or $\log X$ or $\sin X$ we can say sine of X .

Different transformations we can try depending on the visualization that you are doing but it's more of experimentation that you do.

There's no set rule.

You try different transformations and see which one is working well.

In `psycic` learnt package you have a functionality which allows you to create polynomial features which is a good way of creating non linear models.

So this is the importance of your transformation.

So what we have been talking about here we have been talking about feature creation.

Generate powerful new variables, feature selection, remove insignificant features from the analysis.

Helps you to build better models with less computation and feature transformation.

Reshaping features for clarity and impact.

Now remember not every feature is important for your model.

Some are pure noise, you have to remove them.

Some might not have a direct relationship in form of linearity.

So you'll have to transform to model the non linear relationship.

And this is something that you do as a trial and error method and see which one is



working.

Now think of raw data as Cray and your job is to sculpt it into something valuable.

You will practice selection, encoding, scaling, and dimensionality reduction.

That could be another example of feature engineering.

So these are the common task you will be carrying for carrying out.

When you are working with the data, you might have to select the right feature.

You have to encode that you have gone through scaling.

Reducing is the size of the data.

One method I told you is find out what's important, what's not.

Another could be a more statistical approach where you try to see how data can be reduced into lower dimensions.

But these are the aspects that you need to learn.

Now this is this isn't just a theory.

You will apply nasty proven techniques like filter and wrapper methods.

You can use some statistical techniques and model based approaches to identify most predictive features in your data set.

You will go through some.

You have to understand some encoding techniques like how do you encode nominal data which is very common one that we do dummy variable creation that you must have gone through.

Then how do you encode ordinal data?

Ordinal data is such data which has an order and you have to encode it like low, medium and high.

When you talk about the data has some order but the difference between 2 values have no meaning and I talk about low minus medium it does not have a meaning but it has an order.

So that data when you are encoding what approach you will choose or when you are choosing nominal data how you gonna encode them And then you have to also be ready to scale your data.

You can do min Max normalization or Z square standardization or sometimes as I mentioned, you reduce the size of the data and you can use some statistical technique for that like PCA or it could be LDA.

And these techniques help you to reduce the size of the data.



Artificial Intelligence and Machine Learning

That will also be part of your feature engineering techniques.

Now, a great model starts with great features.

So it's your turn to build, test and learn by doing.

So going forward, always focus upon that whatever is given to you.

You don't have to use everything in your model.

You have to be very careful in selecting the right features.

Think about how to transform features which are more suitable for your machine learning model and that will comprise of your feature encoding, transformation and creation.

So on that note, we'll stop here and we'll explore more in our sync sessions.